Rider App (Frontend and Android Core): Developer 3 Handover

Purpose and scope

The rider app lets a rider start a shift, take stock from a hub, accept offers, pack while stopped, drive to a gate, and hand over the order. Developer 3 owns the whole rider app and its Android native core.

NOTE: These features,screens,architecture etc all the things are decided by the information shared with the product scope if it is changed then the endpoints and some things might change…

In scope

- Every rider screen: login and verification status, shift, node, stock, offers, stop-and-pack, navigation, arrival and handover, restock, problems, earnings.
- The Android native core: foreground service, location loop, speed signal, offline event queue.
- The bridge between Flutter and the native core.
- The offline sync of rider events.
- Tests, device trials and release builds for Android.

Out of scope for this doc

- Design and visual styling (the designer's track). Developer 3 styles screens from the design tokens only.
- Backend, dispatch, tracker service and restock logic (Developer 1).
- Customer app and partner tablet app (separate docs).
- iOS work, unless iPhones join the fleet (see open points).
- DigiLocker verification: use a mock until DigiLocker approval (later legal item). The status screen is built now.

Rider app

Feature	What the person does or sees	Why it exists
Login and document status	Signs in, sees what is verified	Only approved riders deliver
Shift start and end	Starts and ends a shift	Pay and safety records
Go online at a node	Told which staging node to wait at and how full it is	Spreads scooters out
Stock screen	Scans lots; sees counts, temperatures, expiry	Stock that matches reality
Offer preview	Sees the possible order, quantity and gate	Informed choice

Feature	What the person does or sees	Why it exists
Stop-and-pack step	Stops safely, packs, presses Ready	No food handling while moving
Safe navigation	Route to the gate with no pressure messages	Safety over the clock
Arrival	Auto-detected, confirmed by QR or beacon scan	GPS alone is not proof
Handover	OTP and quantity check	Complete, proven delivery
Restock	Goes to the hub, takes a queue slot, scans in a fresh batch	Stock continuity
Relocation nudges	Move to Node 7: 3 orders expected, nobody nearby	Better coverage
Earnings	Per order, incentives, paid repositioning time	Fair pay
Problem buttons	Breakdown, accident, gate closed, customer absent, unsafe route	Fast help, no penalty
Offline mode	Events saved and sent when signal returns	Gates often have poor signal
Suggested restock	Sees a drafted request such as 4 burgers from Hub 2; confirms, edits the quantity or declines	The right stock without guessing
Manual restock request	Creates a request: picks items, quantities and a hub	The rider knows about spoiled or damaged units

Words used in these docs

Term	Plain meaning
Gate	The exact entry point the customer picks for delivery
Gate pin	The gate's map location plus entry instructions
Sweet spot	The area a scooter can start from and still reach a gate within the time budget
Staging node	A legal place where a scooter waits for orders
Lot	One labelled batch of food with its prepared and expiry times

Term	Plain meaning
Reservation	A short hold on units so nobody else can buy them
Acceptance	The moment the server confirms the order and starts the 90 s clock
Leg	One scooter's share of an order
Hub	A small site where scooters restock
Kill switch	A control that turns the 90 s offer off for a zone
p90	The time that 9 out of 10 trips beat

Backend needed before you start

Developer 1 delivers these in week 1. Until the real backend exists, build against the mock server.

1. OpenAPI contract

- Covers every endpoint in the API section below.
- Every response includes server_time, so the app can detect a wrong device clock.
- Repeatable writes require an Idempotency-Key header.
- The event batch endpoint (offline sync) is agreed with Developer 3 before the offline queue is built.

2. Mock server (Prism or equivalent)

- Answers every endpoint with schema-valid data.
- Scenario header X-Mock-Scenario selects the state to return.
- Scenarios the app must handle: offer arrives; offer expires before answer; offer taken by another rider; rider moves while stop-and-pack is open (Ready must stay locked); route to gate; arrival rejected (GPS only, QR or code missing); handover with wrong OTP; partial quantity handed over; restock suggestion present; restock confirmed twice (second must not create a request); hub short on a request; verification pending; verification failed; shift limit reached; network down during handover.

3. Ping and event stubs

- The location ping endpoint accepts pings and returns 202, so the spike and the native core can run before the tracker exists.
- The event batch endpoint returns accepted and rejected event IDs, so the offline queue can be tested.

4. Fixtures

- Test riders with a verified and an unverified status.
- Staging nodes, lots with expiry and temperature values, hub stock, and sample gates in the pilot area.

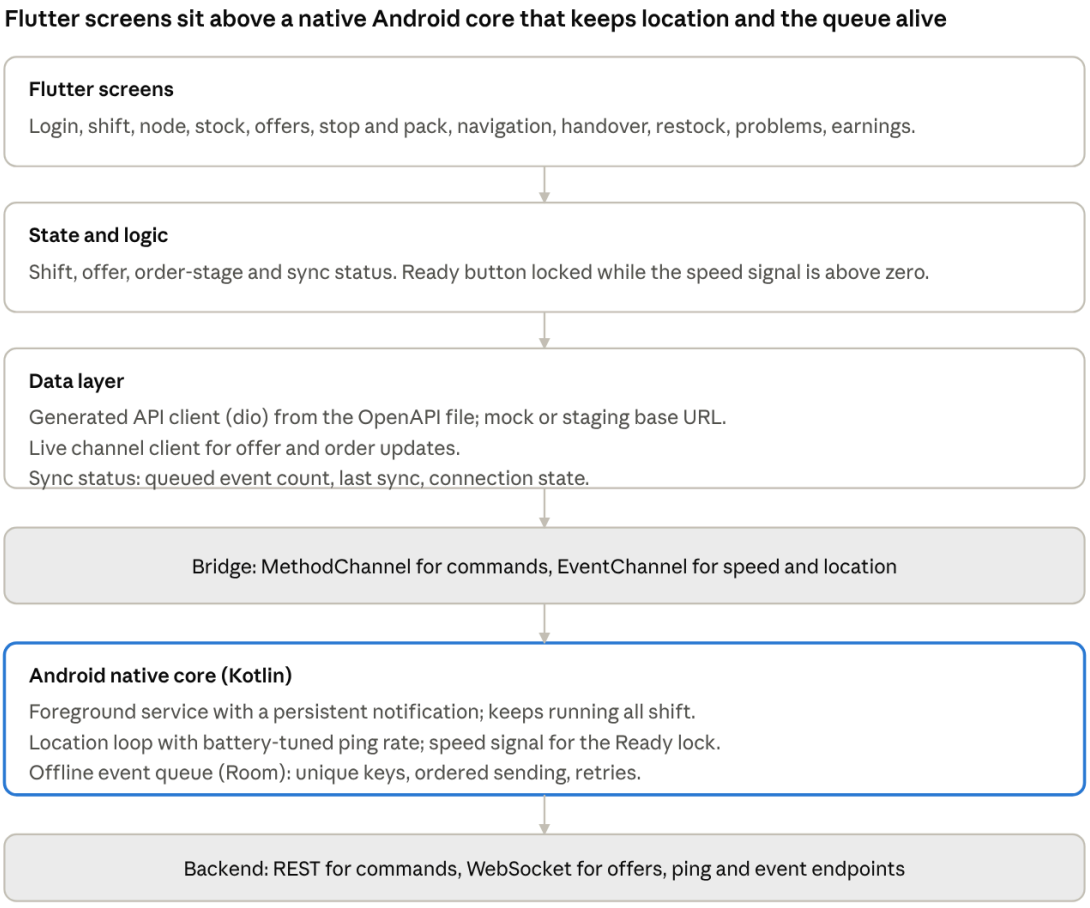
5. Environments

- Mock URL for development, stable for the build.
- Staging URL from Phase 2 for real device trials.

What to do while waiting

- Run the background-location spike first; it needs only the ping stub.
- Build login, shift, node, stock and offer screens on the mock.
- Build the offline queue and the Ready lock against the event and speed stubs.

Architecture



rider app layers · Flutter screens over an Android native core

Flutter draws the screens. The Android core keeps location, the foreground service and the offline queue running even when the screen is off. Screens never call the server directly; the data layer does.

Tech stack and libraries

Flutter is decided for screens. The native core is Kotlin. The background-location spike can change the split; items marked Verify must be checked on the rider phones.

Flutter (screens and state)

Need	Choice	Status
Framework	Flutter, same version as the customer app	Decided
State	Riverpod or Bloc (same choice as the customer app)	Open
Routing	go_router	Proposed
HTTP	dio	Proposed
API client	Generated from OpenAPI	Proposed
Live channel	web_socket_channel	Proposed
Scanning	mobile_scanner	Verify on rider phones
Map display	maplibre_gl and google maps api	Verify
Push	firebase_messaging	Proposed
Crash reports	sentry_flutter	Proposed
Secure token storage	flutter_secure_storage	Proposed
Beacon scanning (only if beacons are chosen)	flutter_blue_plus	Only if chosen

Android native core (Kotlin)

Need	Choice	Status
Location	Google Play services fused location	Proposed; spike first
Foreground service	Android foreground service with notification	Decided
Background retries	WorkManager	Proposed
Offline queue	Room	Proposed
Network	OkHttp	Proposed
Concurrency	Kotlin coroutines	Decided
Bridge to Flutter	MethodChannel (commands), EventChannel (speed, location)	Decided

iOS (only if iPhones are in the fleet)

Need	Choice	Status
Location	CoreLocation with background location mode	Only if iPhones
Background work	BackgroundTasks	Only if iPhones
Offline queue	GRDB or Core Data	Only if iPhones; pick one

Rules

- Pin the Flutter and Dart versions in the repo.
- The native core owns location, the service and the queue. Flutter never owns them.
- The rider app does not use a Go or Node runtime; all server logic is in the backend.

Tools

- Source and CI: GitHub with GitHub Actions. Runs analyze, unit tests, Android unit tests and the debug build on every pull request.
- Mock and contract: Prism mock server; Schemathesis contract tests against the OpenAPI file.
- API manual testing: Bruno or Postman, with the shared collection.
- Device trials: two budget Android phones from the company fleet, mounted on a scooter for the stop-and-pack test.
- Battery and location logging: Android Studio profiler, plus a simple ping log written to a file during trials.
- Design handoff: Figma, read-only; tokens exported as files.
- Test builds: Google Play internal track.
- Crash reports: Sentry.
- Task tracking: the team's chosen tracker (Jira or Linear; pick one).

Screens and tasks

Each screen lists its data, the states it must handle, and when it is done. Native core tasks are listed after the screens.

R1. Project setup

- Navigation, theme from tokens, generated API client, environment switch (mock, staging, production).
- Done when: the app starts on the mock and shows a placeholder home.

R2. Login and verification status / register

- Data: POST /v1/auth/otp/request; POST /v1/auth/otp/verify; GET /v1/riders/me; GET /v1/riders/me/verification; POST /v1/riders/me/verification/start (mock until DigiLocker).
- States: wrong code; resend limit; verification pending; verification failed (show the manual upload path); verification expired.
- Done when: a rider signs in on the mock and sees the correct verification status.

R3. Shift start and end

- Data: POST /v1/riders/me/shifts/start; POST /v1/riders/me/shifts/end.
- States: shift limit reached; a shift already open; network down at start.
- Done when: starting a shift starts the foreground service and ending it stops it.

R4. Online at a staging node

- Data: GET /v1/nodes?zone_id=. Shows node name and how full it is. Shows a relocation suggestion when one exists.
- States: node full; no nodes in zone; relocation suggestion accepted or ignored.
- Done when: the rider sees the node to wait at, and a relocation prompt does not block the screen.

R5. Stock scan

- Data: POST /v1/riders/me/stock/scan; GET /v1/riders/me/stock.
- Shows lot, expiry and temperature for each unit.
- States: bad scan; expired lot (rejected); temperature alert (quarantine prompt); lot already on the scooter.
- Done when: scanned lots match the list returned by the backend.

R6. Offer preview: accept or decline

- Data: GET /v1/riders/me/offers; POST /v1/offers/{id}/accept; POST /v1/offers/{id}/decline. Offers arrive on the live channel.
- Shows items, quantity and gate.
- States: offer expired; offer taken by another rider; accept sent twice (no second accept).
- Done when: accept and decline each produce one server change.

R7. Stop-and-pack

- Data: POST /v1/offers/{id}/ready. Speed comes from the native core (N3).
- Ready is disabled while the speed is above the stop threshold, and shows why.
- States: rider moves while packing; offer cancelled during packing; network down when Ready is pressed.
- Done when: Ready cannot be pressed while moving, on the rider phones.

R8. Navigation to gate

- Data: GET /v1/orders/{id}/route. Map from MapLibre; route line from the backend.
- States: GPS drift; road closed (route refresh); weak signal (keep the last route on screen).
- Done when: the route keeps working on a weak signal and shows no pressure messages.

R9. Arrival and handover

- Data: POST /v1/orders/{id}/arrive; POST /v1/orders/{id}/handover. Arrival needs a scan or code, not GPS alone.
- Handover shows the quantity check and the customer OTP field.
- States: no network (queue the event); wrong OTP; partial quantity; arrival rejected.
- Done when: handover works offline and syncs later, with one confirmed record.

R10. Restock

- Data: GET /v1/riders/me/restock-suggestions; POST /v1/restock/{id}/confirm; POST /v1/restock/{id}/decline; POST /v1/restock (manual request).
- Shows the hub queue slot and the rider's arrival time.
- States: suggestion expired; hub out of stock; restock already under way; confirm tapped twice.
- Done when: confirming makes one request and shows the hub's progress.

R11. Problem buttons

- Data: POST /v1/riders/me/problems.
- Options: breakdown, accident, gate closed, customer absent, unsafe route.
- States: no network (queue it).
- Done when: support sees the report within seconds of sending.

R12. Earnings

- Data: GET /v1/riders/me/earnings.
- Shows orders, incentives, and paid repositioning time.
- Done when: totals match the backend response for the day.

R13. Sync status

- Shows the queued event count, last successful sync, and connection state.
- Done when: a rider can see that events are waiting during an offline period.

N1 to N6. Android native core

- N1 Foreground service: persistent notification, starts on shift start, stops on shift end. Done when it survives the app being swiped away from recents.
- N2 Location loop: pings at a battery-tuned rate, sent to POST /v1/riders/me/location. Done when a full simulated shift keeps pings flowing.
- N3 Speed signal: sent to Flutter over EventChannel for the Ready lock. Done when the Ready lock reacts within two seconds of movement.
- N4 Offline queue: Room table with unique event IDs; sent in order through POST /v1/riders/me/events; retried with backoff. Done when 20 minutes offline syncs in order with no duplicates.
- N5 Gate proximity: device-side geofence with two-ping hysteresis. Done when it does not flip on GPS jitter.
- N6 Bridge: MethodChannel for commands and EventChannel for speed and location. Done when Flutter restarts without stopping the service.

API endpoints the app calls

All paths are under /v1. Every request sends Authorization: Bearer <token>, except the OTP endpoints. Every response includes server_time. Errors use one shape: code, message, retryable (true or false). Every repeatable write sends an Idempotency-Key header.

Area	Method and path	Request key fields	Response key fields	Used in
Auth	POST /v1/auth/otp/request	phone	request_id, expires_in_s	R2
Auth	POST /v1/auth/otp/verify	request_id, otp	access_token, refresh_token, rider_id	R2
Auth	POST /v1/auth/refresh	refresh_token	access_token, refresh_token	all
Profile	GET /v1/riders/me	none	rider (id, name, status)	R2
Verification	GET /v1/riders/me/verification	none	status (pending, verified, failed, expired), reason	R2
Verification	POST /v1/riders/me/verification /start	method	redirect_url or upload_url	R2

Area	Method and path	Request key fields	Response key fields	Used in
Shift	POST /v1/riders/me/shifts/start	node_id	shift (id, started_at)	R3
Shift	POST /v1/riders/me/shifts/end	none	shift (id, ended_at)	R3
Nodes	GET /v1/nodes?zone_id=	zone_id	nodes[] (id, name, fullness_pct)	R4
Stock	POST /v1/riders/me/stock/scan	lot_code, scanned_at	unit (lot_id, expiry, temp_status)	R5
Stock	GET /v1/riders/me/stock	none	lots[] (lot_id, qty, expiry, temp_status)	R5
Offers	GET /v1/riders/me/offers	none	offers[] (id, items, qty, gate, expires_at)	R6
Offers	POST /v1/offers/{id}/accept	none	offer (state), order_id	R6
Offers	POST /v1/offers/{id}/decline	reason	offer (state)	R6
Offers	POST /v1/offers/{id}/ready	none	order (state)	R7
Route	GET /v1/orders/{id}/route	none	polyline, eta_source	R8
Arrival	POST /v1/orders/{id}/arrive	evidence (scan_code or qr), gps, client_time	order (state)	R9
Handover	POST /v1/orders/{id}/handover	otp, quantities[]	order (state)	R9
Restock	GET /v1/riders/me/restock-suggestions	none	suggestions[] (id, hub, items, expires_at)	R10
Restock	POST /v1/restock/{id}/confirm	quantity_override (optional)	request (state)	R10
Restock	POST /v1/restock/{id}/decline	reason	request (state)	R10
Restock	POST /v1/restock	hub_id, items[]	request (id, state)	R10
Problems	POST /v1/riders/me/problems	type, order_id (optional), note	report (id)	R11
Earnings	GET /v1/riders/me/earnings	date	orders, incentives, repositioning_seconds	R12

Area	Method and path	Request key fields	Response key fields	Used in
Location	POST /v1/riders/me/location	lat, lng, speed_mps, accuracy_m, device_time	202 Accepted	N2
Sync	POST /v1/riders/me/events	events[] (event_id, type, payload, device_time)	accepted_ids[], rejected[] (id, reason)	N4, R13
Live	WS /v1/riders/me/stream	token in first frame	offer, order_state, restock, server_time	R6, R7

Notes

- eta_source is measured, placeholder or unavailable. The rider sees the same wording rules as the customer app.
- Arrival and handover send evidence and device time; the server sets the official time.
- The event batch is applied in the order given, and a repeated event_id is accepted without a second effect.
- Field names are a starting point. Developer 1 and Developer 3 agree final names in the first contract review.

Location, offline and ready-lock rules

These rules decide whether a shift is reliable. They are not optional.

1. **Keep the service alive.** The foreground service runs for the whole shift, with a persistent notification. Flutter never owns location.
2. **Ping at a battery-tuned rate.** Pings go to POST /v1/riders/me/location. Record the rate used for each trial.
3. **Ready is locked while moving.** The native speed signal disables Ready and the pack buttons above the stop threshold. The rider sees why it is disabled.
4. **Arrival needs evidence.** GPS alone never confirms arrival. A scan or code is required, and the server sets the official arrival time.
5. **Queue every event.** Arrival, handover, problems and restock confirms are saved in Room with a unique event_id first, then sent in order.
6. **Sync safely.** A repeated event_id is accepted without a second effect. Retry with backoff; never drop an event silently.
7. **Show connection state.** The rider always sees how many events are waiting and when the last sync worked.
8. **Offers and order changes come from the server.** A stale offer shows as expired, not as accepted.
9. **Handover is confirmed once.** A repeated tap or retry must not create a second handover.

Testing and release checklist

No build goes to testers until every item is ticked. The shift test is the gate for the whole rider design.

- ☐ Background-location spike: a full simulated shift on two budget Android phones, with pings unbroken throughout.
- ☐ Battery use measured over the shift and recorded in the doc.
- ☐ Foreground service survives the app being swiped from recents.
- ☐ 20 minutes offline: events sync in order with no duplicates.
- ☐ Ready lock tested while walking and while on the scooter.
- ☐ Arrival rejected on GPS alone; accepted with scan or code.
- ☐ Handover tested with wrong OTP, partial quantity and no network.
- ☐ Restock confirm tapped twice creates one request.
- ☐ Accept tapped twice creates one acceptance.
- ☐ Unit tests for the queue, geofence hysteresis and speed threshold pass.
- ☐ Widget tests for every screen state in the screen list pass.
- ☐ Integration tests against the mock server pass in CI.
- ☐ Contract tests against the Developer 1 OpenAPI file pass.
- ☐ Text readable in sunlight; tap targets large enough for gloves.
- ☐ Screen-reader labels on every control.
- ☐ No new crash in the last build in Sentry.
- ☐ Build shipped to internal testers before any real rider is on the fleet.

Open points

These are not decided yet. Developer 3 can make the choice listed, record it in the doc, and tell Developer 1 if it changes an endpoint.

- Spike result decides the design: if background location holds on the rider phones, keep the native core as described; if not, the rider app moves fully native and this doc is rewritten.
- Whether iPhones join the fleet. If not, iOS work is removed from scope.
- Beacon or QR for arrival evidence. QR is the default; drop beacons if QR is enough.
- Exact stop threshold for the Ready lock (metres per second). Set it during the trial and record it.
- Battery-tuned ping rate. Set it during the trial and record it.
- Final field names in each response. Agree in the first contract review.
- Wording of every rider message. The copy deck owns the words; the app shows them.
- State library: Riverpod or Bloc, the same choice as the customer app.

If a feature or flow changes during the build, record the change in this doc with the date, the reason, and the developer who made it.